

RESUMO DO MÓDULO

Chain of Responsibility

O Chain of Responsibility (Cadeia de Responsabilidade) é um padrão de projeto que pode ser aplicado em diversas linguagens de programação, inclusive no JavaScript com o Express.js. Esse padrão estabelece uma sequência de objetos encadeados que são responsáveis por tratar uma requisição. A principal ideia é que cada objeto na cadeia possui um comportamento específico e, se não é capaz de lidar com a requisição, ela é passada para o próximo objeto na cadeia.

O Express.js adota esse padrão para lidar com o fluxo de requisições e respostas, utilizando uma série de middlewares que atuam como nós nessa cadeia de responsabilidade. Cada middleware executa uma tarefa específica e decide se a requisição deve seguir para o próximo middleware ou ser encerrada.

Como funciona o Chain of Responsibility no Express?

Quando uma requisição é feita para um endpoint no Express, ela passa por uma série de middlewares antes de chegar ao controlador que irá tratá-la. Cada middleware é responsável por executar uma ação específica, como validação de dados, autenticação, autorização, entre outras. Se o middleware não conseguir lidar com a requisição, ele deve passar o controle para o próximo middleware, utilizando a função `next()`.

Caso contrário, ele pode responder diretamente ao cliente e interromper a cadeia. Se nenhum middleware for capaz de lidar com a requisição, o fluxo termina com um erro ou uma mensagem de que a rota não foi encontrada.

Exemplo de Chain of Responsibility no Express

Suponha que você tenha uma rota em sua aplicação Express que precisa verificar se o usuário que está tentando acessar uma determinada página tem permissão para fazê-lo. Vamos implementar um conjunto de middlewares para verificar permissões em uma ordem específica, até que uma

delas permita o acesso ou todas rejeitem e bloqueiem a requisição.

Aqui está um exemplo prático:

javascript

```
const express = require('express');
```

```
const app = express();
```

```
// Middleware que verifica se o usuário é administrador
```

```
const checkAdmin = (req, res, next) => {
```

```
  if (req.user && req.user.isAdmin) {
```

```
    next(); // passa para o próximo middleware
```

```
  } else {
```

```
    res.status(403).send('Você não tem permissão para acessar esta página!');
```

```
  }
```

```
};
```

```
// Middleware que verifica se o usuário está autenticado
```

```
const checkAuth = (req, res, next) => {
```

```
  if (req.isAuthenticated()) {
```

```
    next(); // passa para o próximo middleware
```

```
  } else {
```

```
    res.status(401).send('Você precisa estar logado para acessar esta página!');
```

```
  }
```

```
};
```

```
// Rota protegida por permissões

app.get('/admin', checkAuth, checkAdmin, (req, res) => {

  res.send('Bem-vindo à página de administração!');

});

app.listen(3000, () => {

  console.log('Servidor em execução em http://localhost:3000/');

});
```

Explicação do Código

- O primeiro middleware, `checkAuth`, verifica se o usuário está autenticado. Caso esteja, ele chama `next()` para passar a requisição para o próximo middleware. Caso contrário, ele envia uma resposta com o código de status 401 (Não Autorizado).
- O segundo middleware, `checkAdmin`, verifica se o usuário autenticado possui permissões de administrador (`req.user.isAdmin`). Se sim, a requisição segue para o próximo middleware. Caso contrário, é enviada uma resposta com o código de status 403 (Proibido).
- A rota `/admin` é protegida por ambos os middlewares, que são executados na ordem em que foram definidos. Se o usuário não estiver autenticado, a resposta será 401. Se o usuário estiver autenticado, mas não for um administrador, ele receberá 403. Se o usuário for autenticado e administrador, ele receberá a resposta de sucesso da rota (Bem-vindo à página de administração!).

Concatenando `res.status()` com `res.send()`

Uma observação importante é que, no Express, é possível encadear métodos como `status()` e `send()` na mesma linha:

javascript

```
res.status(403).send('Você não tem permissão para acessar esta página!');
```

Esse encadeamento torna o código mais limpo e legível, economizando linhas e facilitando a leitura.

Vantagens do Padrão Chain of Responsibility

O uso do padrão Chain of Responsibility permite que você construa um fluxo de execução flexível e modular para tratar requisições. Cada middleware representa um ponto de verificação ou uma ação específica a ser realizada. Isso ajuda a:

- Organizar o código: Mantendo cada responsabilidade separada em seu próprio middleware.
- Facilitar a manutenção: Você pode adicionar ou remover middlewares na cadeia sem precisar reescrever a lógica existente.
- Reutilizar lógica: Middlewares podem ser usados em diferentes rotas, compartilhando funcionalidades como autenticação, verificação de permissões e muito mais.

Conclusão

O padrão Chain of Responsibility é uma poderosa abordagem para gerenciar fluxos de requisições no Express.js. Ao utilizar middlewares, você cria um processo ordenado e controlado para lidar com cada requisição, passando o controle de um middleware para outro até que a resposta seja enviada ao cliente. Isso proporciona um controle granular e uma melhor estruturação de aplicações web.
